

Stop When It Matters: Detectability-Guided Microbenchmarking for Performance Regression Testing

Zongxiong Chen

Fraunhofer FOKUS
Germany

Derui Zhu*

Rochester Institute of Technology
USA

Jinfu Chen

Wuhan University
China

Kundi Yao

Ontario Tech University
Canada

Alexander Pretschner

Technical University of Munich &
fortiss
Germany

Weiwei Shang

University of Waterloo
Canada

Manfred Hauswirth

Fraunhofer FOKUS & Technische
Universität Berlin
Germany

Sonja Schimmler

Fraunhofer FOKUS & Technische
Universität Berlin
Germany

Abstract

Performance microbenchmarks are widely used for detecting performance regressions in continuous integration, yet existing execution policies remain computationally expensive. We present detectability-guided microbenchmarking, a principled framework that reformulates benchmark execution as a Minimum-Detectable-Effect (MDE) problem, achieving substantial speedups with formal statistical guarantees. Our approach introduces a two-threshold specification: a lower threshold, δ_0 , for negligible slowdowns, and an upper threshold, δ_1 , for meaningful regressions. We theoretically and empirically show that this specification enables bounded false-positive rates below δ_0 and guaranteed statistical power around δ_1 , while dramatically reducing execution time. To realize this framework, we propose a two-phase procedure: Phase 1 adaptively controls execution via online sample-size re-estimation guided by minimum detectable effect, enabling *early stop* when sufficient statistical resolution is reached and *futile abort* when target detectability is unattainable within the execution budget; Phase 2 performs regression analysis under the same specification, yielding decisions with provable error bounds. Experiments on 1,702 microbenchmarks from three large-scale open-source projects demonstrate up to 93% reduction in execution time while empirical false-positive rates and power closely match their theoretical targets. Our findings reveal that detectability-guided execution offers a theoretically grounded yet practical path toward efficient microbenchmark-based regression detection.

CCS Concepts

• Software and its engineering → Software maintenance tools;

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2882-2/2026/10

<https://doi.org/10.1145/3832783.3834367>

Keywords

Performance Regression Detection, Adaptive Performance Benchmarking, Minimum Detectable Effect

ACM Reference Format:

Zongxiong Chen, Derui Zhu, Jinfu Chen, Kundi Yao, Alexander Pretschner, Weiwei Shang, Manfred Hauswirth, and Sonja Schimmler. 2026. Stop When It Matters: Detectability-Guided Microbenchmarking for Performance Regression Testing. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3834367>

1 Introduction

Performance microbenchmarks are widely used to detect regressions in software and are increasingly integrated into continuous integration (CI) pipelines[14, 16, 19]. Frameworks such as Java Microbenchmark Harness (JMH)[30] enable fine-grained version comparisons, but in CI environments, benchmarks may run repeatedly across many commits, making execution efficiency critical. Despite this need, policies that govern benchmark execution duration remain largely heuristic [18, 26]: developers either allocate fixed time budgets or rely on stability-based stopping rules that terminate once measurements on collected data appear stable.

However, these policies do not explicitly consider whether the collected data are sufficient for reliable regression detection. Statistical stability does not imply detectability: a benchmark may exhibit low variance yet lack the statistical power to detect a meaningful regression. Conversely, a noisy benchmark may already provide enough evidence to identify regressions, yet stability-based criteria continue execution unnecessarily. As a result, execution effort is often misaligned with the actual goal of regression testing—reliably identifying meaningful performance changes while controlling both false positives and false negatives.

To address this challenge, we propose **detectability-guided microbenchmarking**, a framework that controls benchmark execution based on regression detectability rather than measurement stability. The key idea is to ask whether the collected data suffice to detect a regression of practical interest with a false-positive rate

at most α and statistical power at least $1 - \beta$. To formalize this, we introduce a *two-threshold MDE specification* (δ_0, δ_1) that distinguishes negligible slowdowns from meaningful regressions: the lower threshold δ_0 bounds false positives for changes too small to matter, while the upper threshold δ_1 ensures adequate power to detect regressions of practical concern. This explicit indifference region enables statistically grounded regression decisions. Building on this specification, we design an adaptive two-phase execution framework comprising *measurement acquisition* (Phase 1) and *regression analysis* (Phase 2). In Phase 1, online sample-size re-estimation [33] dynamically controls execution: the benchmark stops early once sufficient statistical resolution is reached, or aborts when reliable detection is infeasible within the given budget (*futile abort*). In Phase 2, regression decisions are made under the same two-threshold specification, yielding bounded false-positive rates below δ_0 and target power around δ_1 .

To the best of our knowledge, this is the first approach that uses Minimum-Detectable-Effect (MDE) as an online, adaptive execution-control microbenchmark for performance regression detection. Unlike the common stability-based heuristics, our approach directly aligns execution effort with regression-detection objectives and provides theoretical guarantees on the detection quality. We implemented this approach in JMH and evaluated it on 1,702 microbenchmarks from three widely used Java projects (RxJava, Eclipse Collections, and Zipkin). The results show execution-time reductions of up to 93% while maintaining bounded error rates that match their configured targets.

Overall, this work makes the following contributions:

- We introduce detectability-guided microbenchmarking, which reframes benchmark execution control from achieving measurement stability to ensuring regression detectability, enabling order-of-magnitude efficiency gains.
- We introduce two metrics, *Adequate Stop* and *Underpowered Non-Regression*, enabling principled evaluation of microbenchmark stopping decisions by quantifying whether executions achieve sufficient statistical power for reliable regression detection.
- We conduct a large-scale empirical evaluation demonstrating that our approach substantially reduces overall execution time while achieving high statistical power to detect regressions and lower false-positive rates compared to existing methods.
- Our study shows that the regression detection can be bounded by the target detection resolution, providing practitioners with a new lens for reasoning about efficiency in benchmark design.

Paper organization: Section 2 reviews related work. Section 3 presents a preliminary study showing why stability alone does not imply regression detectability. Section 4 introduces the detectability-guided methodology and the TT-MDE framework. Section 5 describes the study setup. Section 6 reports the results. Section 8 discusses threats to validity, and Section 9 concludes the paper.

2 Related Work

Microbenchmarking Methodology. Prior work established statistical foundations for reliable performance evaluation. Georges et al. [13] and Kalibera and Jones [21] analyzed sources of variance and provided guidelines for rigorous benchmarking. Mytkowicz et al. [28] and Curtsinger and Berger [9] addressed measurement

bias caused by environmental factors such as memory layout. These works improve measurement validity and reproducibility, but they typically assume fixed sampling budgets and do not address how measurement effort should adapt during execution to meet regression detection objectives.

Adaptive Measurement Acquisition. Several approaches reduce benchmarking cost by stopping measurements once results appear stable. Laaber et al. [26] dynamically reconfigure microbenchmarks based on distribution similarity, and Alghmadi et al. [1] propose early stopping rules based on repetitive measurement trends. These techniques reduce redundant measurements by exploiting measurement stability, but they do not reason about effect size or detectability. In contrast, our framework treats benchmarking as a regression detection task and uses detectability via minimum detectable effect as the stopping signal, explicitly targeting regressions.

Regression Analysis and Detection. Another line of research focuses on detecting regressions after measurement data have been collected. Huang et al. [16], Mostafa et al. [27], and Chen et al. [4] study regression detection and prioritization. Industrial systems such as ServiceLab [6] analyze large-scale production telemetry to identify performance anomalies. These approaches improve regression identification but generally treat measurement acquisition as fixed. Consequently, the processes of collecting measurements and detecting regressions remain largely decoupled.

Sequential and Adaptive Designs. Sequential and adaptive experimental designs allow experiments to stop early while controlling statistical error rates [32, 38, 42]. Such designs generally support two types of early termination: Stopping when sufficient evidence has been collected (*early stop*), and aborting experiments when achieving the desired objective becomes unlikely (*futile abort*). Futile abort is widely used in domains such as clinical trials [3, 7, 12, 40, 44], where designs such as Simon’s two-stage design [39] include explicit early termination rules to reduce the expected sample size when treatments are ineffective. While these approaches provide strong statistical foundations for efficient experimentation, they have not been integrated into software microbenchmark execution workflows. Our work adapts the concept of *futile abort* to benchmarking by defining futility in terms of regression detectability and by using variance-based detectability estimates to guide execution decisions.

Summary. Table 1 summarizes the differences between our work and prior approaches based on the main properties. Existing techniques either focus on measurement stability, measurement reliability, or regression detection independently. In contrast, our framework integrates measurement acquisition and regression analysis through a unified statistical perspective. Specifically, we use the minimum detectable effect as an online execution-control signal, enabling **detectability-guided adaptive sampling**, and *futile abort* when meaningful regression detection becomes infeasible.

3 A Preliminary Study: Stability Does Not Imply Detectability

Microbenchmark practice often assumes that once measurements appear statistically stable, further execution is unnecessary. Stability metrics such as the coefficient of variation (CV) are widely used

Table 1. Comparison with Existing Work. Phase 1: Measurement Acquisition; Phase 2: Regression Analysis.

Work	Target Phase	Execution Context	Stopping Signal	Online	Adaptive	Early Stop	Futile Abort	Effect Size Aware	Power Aware
Fixed-budget benchmarks	Phase 1	Microbenchmark	/	✗	✗	✗	✗	✗	✗
Laaber et al. [26]	Phase 1	Microbenchmark	CV/CI/KLD	✓	✓	✓	✗	✗	✗
Alghamdi et al. [1]	Phase 1	Whole system	Repetitiveness	✓	✓	✓	✗	✗	✗
μ OpTime [18]	Phase 1	Microbenchmark	CV/CI	✗	✓	✓	✗	✗	✗
STABILIZER [9]	Phase 2	Whole system	Variance	✓	✗	✗	✗	✗	✗
Laaber et al. [25]	Phase 2	Microbenchmark	/	✗	✗	✗	✗	✓	✓
PeASS [35]	Phase 2	Microbenchmark	/	✗	✗	✗	✗	✓	✓
ServiceLab [6]	Phase 2	Whole system	/	✗	✗	✗	✗	✓	✓
FBDetect [45]	Phase 2	Whole system	/	✓	✗	✗	✗	✓	✗
Kalibera and Jones [21]	Phase 1 & Phase 2	Microbenchmark	Variance/CI	✗	✓	✓	✗	✓	✗
Our approach	Phase 1 & Phase 2	Microbenchmark	Detectability	✓	✓	✓	✓	✓	✓

CV: coefficient of variation; CI: confidence interval; KLD: Kullback-Leibler divergence

Table 2. Failure-to-detect rate for injected regressions of 0.5%, 1%, 5%, and 10% among benchmarks selected by a strict stability criterion.

Subject	Failure-to-Detect Rate			
	0.5%	1%	5%	10%
RxJava	81.18%	20.39%	0.30%	0.30%
Eclipse Collections	100.00%	100.00%	75.00%	66.67%
Zipkin	95.24%	46.67%	0.00%	0.00%
Total	82.73%	24.43%	1.15%	0.89%

to decide when results have converged [17, 26]. However, stability reflects within-version consistency, whereas regression testing requires distinguishing two versions whose performance differs by a practically significant amount. Low variance alone does not guarantee sufficient discriminatory power.

To quantify this mismatch, we analyzed microbenchmarks across three Java projects in the preliminary study. We first selected benchmarks whose baseline version satisfies a strict stability criterion ($CV \leq 1\%$) [17]. For these stable benchmarks, we followed the prior regression-evaluation methodology, injecting oracle bug size of 0.5%, 1%, and 10% as described in Section 6.3. We then applied a two-sample statistical test (see Section 4.2) and recorded whether the injected regression was detected. A benchmark is considered to *fail detection* if no statistically significant slowdown is reported.

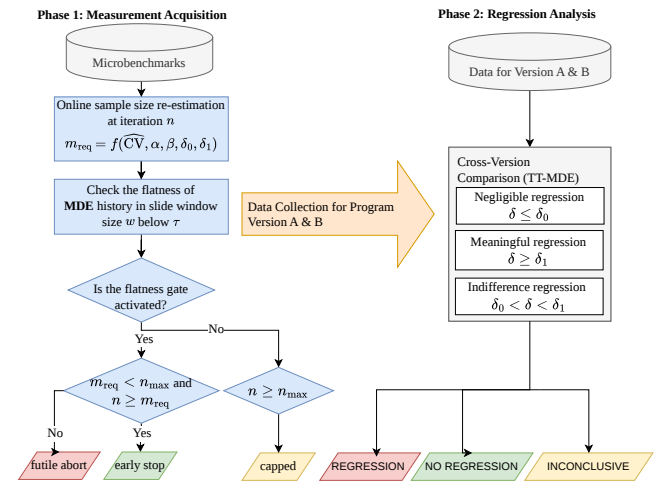
Table 2 reports the failure-to-detect rate among stable benchmarks, indicating stability is a poor proxy for detectability. At a 0.5% target regression, 82.73% of selected benchmarks fail overall, with rates above 80% in all three projects, showing that low CV does not provide sufficient power for very small regressions. Although detectability improves for larger targets, the mismatch remains: Eclipse Collections still fails in 75.00% of cases at 5% and 66.67% at 10%. Overall failure remains 24.43% at 1% and 1.15% at 5%, and even RxJava still fails in 20.39% of cases at 1%.

These results demonstrate that stability does not imply detectability. Benchmarks may appear statistically stable while lacking sufficient power to detect small but meaningful regressions. Moreover, stability-based stopping does not explicitly reason about target effect sizes, and thus cannot guarantee regression sensitivity. Together, these findings reveal a structural misalignment between stability-driven execution and regression testing objectives, motivating the need for a detectability-guided measurement control.

4 Methodology

Performance regression testing aims to determine whether a new program version introduces a meaningful regression. In microbenchmarking, however, measurement noise and execution cost make it unclear how much data are needed for a reliable decision. Existing approaches either stop when measurements appear stable or use fixed measurement budgets, but they rarely determine sampling effort from the ability to detect meaningful regressions.

To address this issue, we propose a two-phase detectability framework that links measurement acquisition and regression analysis through the minimum detectable effect (MDE). We frame regression testing as a MDE problem: rather than asking whether any statistically significant difference exists, we ask whether the slowdown exceeds a meaningful threshold (e.g., 1% or 5%). As illustrated in Figure 1, **Phase 1 (measurement acquisition)** executes each program version separately to collect measurements and estimate variance. Sampling continues until the target statistical resolution is reached or the budget indicates that the target detectability is unattainable. **Phase 2 (regression analysis)** then compares the collected measurements to determine whether the new version exhibits a meaningful regression relative to the original version. Both phases use the same two-threshold specification to distinguish negligible slowdowns from meaningful regressions.

**Figure 1. Illustration of two-phase detectability-guided adaptive benchmarking workflow.**

Algorithm 1: Measurement Acquisition in TT-MDE

Input: benchmark b ; max forks $f_{\max}$; max iterations per fork $i_{\max}$;
 window size w ; flatness gate threshold τ ; budget
 $n_{\max} = f_{\max} \cdot i_{\max}$ (or fork budget for MNF); error targets
 (α, β) encoded as $Z = z_{1-\alpha} + z_{1-\beta}$; regression thresholds:
 TT-MDE with (δ_0, δ_1)

Output: samples S (in JMh arrival order); state
 $\in \{\text{early stop, futile abort, capped}\}$

```

1  $S \leftarrow \emptyset$ ; MDE history  $H \leftarrow \emptyset$ ;
2 for  $f \leftarrow 1$  to  $f_{\max}$  do
3   init tracker state for fork  $f$ ;
4   for  $i \leftarrow 1$  to  $i_{\max}$  do
5      $x \leftarrow \text{EXECUTEONEITERATION}(b, f, i)$ ;
6     Append( $S, x$ );
7     // Compute effective sample size and required
      size (EffN defined in Section 4.3)
       $\widehat{CV} \leftarrow CV(S)$ ;  $m_{\text{eff}} \leftarrow \text{EFFN}(S)$ ;  $m_{\text{req}} \leftarrow \left\lceil \frac{2\widehat{CV}^2 Z^2}{(\delta_1 - \delta_0)^2} \right\rceil$ ;
8     // Update MDE history for flatness monitoring
       $\widehat{MDE} \leftarrow Z\sqrt{2} \cdot \widehat{CV} / \sqrt{m_{\text{eff}}}$ ;
9     Append( $H, \widehat{MDE}$ );
10    if  $\text{IsFLAT}(H, w, \tau)$  then // gate decisions on
      flatness over last  $w$  checkpoints
11      if  $m_{\text{eff}} \geq m_{\text{req}}$  then // early stop when
        effective sample size is sufficient
12        return ( $S, \text{early stop}$ );
13      else
14        // futile abort if required size stays
        above maximum execution budget
15        if  $m_{\text{req}} > n_{\max}$  then
          return ( $S, \text{futile abort}$ );
16  // continue to next fork unless stopped
17 return ( $S, \text{capped}$ );
```

4.1 Phase 1: Detectability-guided Measurement Acquisition with TT-MDE

Phase 1 performs sequential measurement acquisition via sample size re-estimation [22, 43] guided by detectability (see Algorithm 1). For a given version v , let $S_v = (x_{v,1}, \dots, x_{v,n})$ denote the measurements collected so far in the arrival order of JMh with running mean $\widehat{\mu}_v$ and standard deviation $\widehat{\sigma}_v$. Under a normal approximation, the standard single-version MDE for a single sample of size n is given as: $\text{MDE}_v^{\text{abs}} = (z_{1-\alpha} + z_{1-\beta}) \cdot \widehat{\sigma}_v / \sqrt{n}$, where α is the target false-positive rate, $1 - \beta$ is the nominal target power, and $z_{1-\alpha}$ and $z_{1-\beta}$ are the corresponding quantiles of the standard normal distribution respectively. This quantity is expressed on the absolute measurement scale.

However, TT-MDE does not reason about absolute differences. The downstream decision in Phase 2 concerns *relative* regression effects, bounded by the two thresholds (δ_0, δ_1) . To express detectability on the same scale, we normalize by the mean and obtain the relative MDE: $\text{MDE}_v^{\text{rel}} = (z_{1-\alpha} + z_{1-\beta}) \cdot \frac{\widehat{\sigma}_v / \widehat{\mu}_v}{\sqrt{n_v}} = (z_{1-\alpha} + z_{1-\beta}) \cdot \widehat{CV}_v / \sqrt{n_v}$, where $\widehat{CV}_v = \widehat{\sigma}_v / \widehat{\mu}_v$ is the coefficient of variation. This normalization is necessary because the practical decision boundaries δ_0 and δ_1 are themselves defined as relative effects.

Although Phase 1 collects measurements for one version at a time, the required statistical resolution is induced by the *cross-version* regression decision performed in Phase 2.

Under the symmetric approximation used by TT-MDE, the standard error of the cross-version contrast is approximately $\sqrt{2}$ ¹ times the single-version standard error, yielding the operational detectability proxy $\widehat{\text{MDE}}_v \approx \sqrt{2}(z_{1-\alpha} + z_{1-\beta}) \cdot \widehat{CV}_v / \sqrt{m_{\text{eff},v}}$, where $m_{\text{eff},v} = \text{EFFN}(S_v)$ is the effective sample size of the currently collected measurements. The concrete definition of EFFN depends on the selected uncertainty estimator and is given in Section 4.3. Using $m_{\text{eff},v}$ rather than the raw sample count n_v allows Phase 1 to remain compatible with dependence-aware variance estimation.

TT-MDE uses $\widehat{\text{MDE}}_v$ to determine whether the current measurements of version v are sufficiently informative. The key requirement is that the smallest effect reliably distinguishable from noise should not exceed the gap between the negligible-effect boundary δ_0 and the meaningful regression boundary δ_1 . Since this gap equals the width of the indifference region, the stopping criterion is $\widehat{\text{MDE}}_v \leq \delta_1 - \delta_0$. If $\widehat{\text{MDE}}_v > \delta_1 - \delta_0$, then the current uncertainty is too large to resolve the decision gap defined by (δ_0, δ_1) . Phase 2 would not be able to reliably separate negligible effects from meaningful regressions. Requiring $\widehat{\text{MDE}}_v \leq \delta_1 - \delta_0$ therefore ensure that Phase 1 only stops once the measurement precision matches the resolution required by Phase 2.

Solving this criterion for the required effective sample size yields $m_{\text{req},v} \geq \left\lceil \frac{2\widehat{CV}_v^2 (z_{1-\alpha} + z_{1-\beta})^2}{(\delta_1 - \delta_0)^2} \right\rceil$. Thus, the required amount of measurement effort increases with the variance of the version under test and with stricter error-control targets, and decreases as the indifference region becomes wider. For readability, we omit the subscript v when the current version is clear from the context.

Flatness-Gated Detectability Activation. TT-MDE omits an explicit warmup phase and instead uses a *flatness gate* to control when adaptive stopping becomes active. Early iterations are collected unconditionally, and stopping decisions are disabled until a flatness gate is satisfied. Let $\widehat{\text{MDE}}_i$ denote the estimated detectability quantity used for flatness monitoring after the i -th checkpoint, and let $H = \{\widehat{\text{MDE}}_{i-w+1}, \dots, \widehat{\text{MDE}}_i\}$ be the most recent window of size w . We define the flatness gate as $\text{IsFLAT}(H, w, \tau) = \mathbb{I}[\max(H) - \min(H) \leq \tau]$. This gate does not define the stopping target itself. Instead, it serves as a safeguard against transient non-stationarity during warmup-like behavior. First, early instability inflates variance estimates, which in turn keeps $\widehat{\text{MDE}}$ large and prevents premature termination. Second, even if detectability appears temporarily favorable, stopping remains disabled until the recent $\widehat{\text{MDE}}$ trajectory becomes sufficiently flat. Thus, TT-MDE remains detectability-centered, while the flatness gate only determines when stopping decisions are permitted.

Termination States. Phase 1 terminates in one of three states. It returns *early stop* when $m_{\text{eff}} \geq m_{\text{req}}$, indicating that the current

¹ **Origin of the $\sqrt{2}$ factor.** Let s_A and s_B denote the standard errors of the estimated means for versions A and B , respectively. Since the two versions are measured independently, the variance of their difference is $\text{Var}(\widehat{\mu}_A - \widehat{\mu}_B) = s_A^2 + s_B^2$. Under the symmetric approximation $s_A \approx s_B \approx s_v$, the standard error of the cross-version contrast becomes $\sqrt{s_A^2 + s_B^2} \approx \sqrt{2}s_v$.

version has achieved sufficient effective information for the downstream regression analysis. It returns *futile abort* when $m_{\text{req}} > n_{\text{max}}$, where n_{max} is the maximum execution budget under the current run configuration, indicating that the required effective resolution is unlikely to be reached within budget under the current variability estimate. Otherwise, execution continues until a stopping decision is reached or the maximum budget is exhausted (*capped*).

4.2 Phase 2: Regression Decision with an Indifference Region

After Phase 1 completes for both versions, Phase 2 performs a cross-version comparison using the collected measurements. Due to the space limitations, we present the statistical procedure here at a conceptual level; the full pseudo code of Phase 2 is provided in the replication package. Let Δ be the true relative regression of B over A , and let $\hat{\Delta}$ be its estimate. We orient the metric such that $\hat{\Delta} > 0$ indicates a regression. For higher-is-better metrics (e.g., throughput), $\hat{\Delta} = \frac{\bar{\mu}_A - \bar{\mu}_B}{\bar{\mu}_A}$, whereas for lower-is-better metrics (e.g., latency), $\hat{\Delta} = \frac{\bar{\mu}_B - \bar{\mu}_A}{\bar{\mu}_A}$. Thus, TT-MDE applies to any scalar performance metric with a known degradation direction. The subsequent statistical procedure is unchanged. Rather than treating regression detection as a point-null problem, we use the two-threshold specification (δ_0, δ_1) to define detectability with an explicit indifference region. The lower threshold δ_0 is the limit of negligible-effects used in the final decision on regression, whereas the upper threshold δ_1 defines the meaningful performance regression size that the procedure is designed to detect with the target power.

To quantify uncertainty in the performance regression estimate, Phase 2 computes a dependence-aware standard error using the same effective sample size concept employed in Phase 1. Let $\hat{\sigma}_A^2$ and $\hat{\sigma}_B^2$ denote the sample variances of S_A and S_B , and let $m_{\text{eff},A} = \text{EFFN}(S_A)$ and $m_{\text{eff},B} = \text{EFFN}(S_B)$ be the corresponding effective sample sizes. The standard error of the relative performance regression estimate is approximated by $SE \approx \frac{1}{\bar{\mu}_A} \sqrt{\hat{\sigma}_A^2 / m_{\text{eff},A} + \hat{\sigma}_B^2 / m_{\text{eff},B}}$. Using this uncertainty estimate, Phase 2 computes the stopping-time detectability bound $\widehat{\text{MDE}}_{\text{stop}} = (z_{1-\alpha} + z_{1-\beta}) \cdot SE$. If $\widehat{\text{MDE}}_{\text{stop}} > \delta_1 - \delta_0$, then the collected data do not provide sufficient resolution to separate negligible and practically meaningful slowdowns, and the procedure returns *INCONCLUSIVE*. Otherwise, Phase 2 performs the one-sided hypothesis test

$$H_0 : \Delta \leq \delta_0 \quad \text{vs.} \quad H_1 : \Delta > \delta_0.$$

Using the test statistic $T = \frac{\hat{\Delta} - \delta_0}{SE}$, the procedure reports *REGRESSION* when $T > z_{1-\alpha}$ and *NOREGRESSION* otherwise.

This construction yields two complementary design targets. First, for negligible slowdowns, the final one-sided test controls false positives at level α :

$$\Pr(\text{REPORTREGRESSION} \mid \Delta \leq \delta_0) \leq \alpha.$$

Second, whenever the detectability condition is satisfied, the sample size attained is designed to provide approximately the target power $1 - \beta$ for practically meaningful regressions at least δ_1 :

$$\Pr(\text{REPORTREGRESSION} \mid \Delta \geq \delta_1, \widehat{\text{MDE}}_{\text{stop}} \leq \delta_1 - \delta_0) \approx 1 - \beta.$$

The intermediate region $\delta_0 < \Delta < \delta_1$ is intentionally treated as an indifference zone [37], where no sharp guarantee is imposed. In

this region, the probability of reporting a regression is expected to transition smoothly between the false-positive-controlled and adequately powered regimes.

In this way, Phase 2 is aligned with the two-threshold detectability specification established in Phase 1. Phase 1 determines whether sufficient information can be collected to resolve the gap between δ_0 and δ_1 , while Phase 2 makes the final decision using the joint stopping-time uncertainty of both versions.

4.3 Variance Estimation and Dependence Handling

In practice, microbenchmark data often exhibit temporal dependence and hierarchical structure due to JVM warmup, garbage collection, and process forking. To accommodate such effects, we instantiate Phase 1 with alternative variance or effective sample size estimators while preserving the same TT-MDE control logic. The resulting variants are evaluated empirically in Section 6.

Let $\widehat{\text{Var}}(\hat{\mu}_v)$ denote the estimator-specific variance of the sample mean for version v . When convenient, we express this uncertainty through an equivalent effective sample size $m_{\text{eff},v} = \hat{\sigma}_v^2 / \widehat{\text{Var}}(\hat{\mu}_v)$, so that $\widehat{\text{Var}}(\hat{\mu}_v) \approx \hat{\sigma}_v^2 / m_{\text{eff},v}$. All variants differ only in how measurement uncertainty is estimated; the MDE computation, stopping rule, and Phase 2 decision remain unchanged.

Naive MDE (MNP) assumes independent and identically distributed observations. Variance is estimated using the unbiased sample variance, and the nominal sample size is used directly as $m_{\text{eff}} = |S|$. This serves as the simplest baseline estimator.

AR(1) MDE (MAR1) accounts for short-range autocorrelation by estimating the lag-1 coefficient ρ and using the standard AR(1) [41] effective sample size approximation $m_{\text{eff}} = \frac{(1-\rho)}{(1+\rho)} \cdot |S|$. This variant is suitable when dependence is predominantly local and reasonably summarized by first-order autocorrelation.

Newey-West MDE (MNW) [29] uses a heteroskedasticity- and autocorrelation-consistent (HAC) estimator of the variance of the sample mean, denoted $\widehat{\text{Var}}_{\text{NW}}(\hat{\mu})$, to remain robust under more general dependence. For consistency with the TT-MDE interface, we convert this variance estimate into an equivalent effective sample size $m_{\text{eff}} = \frac{\hat{\sigma}^2}{\widehat{\text{Var}}_{\text{NW}}(\hat{\mu})}$.

Fork-Level MDE (MNF) combines iteration-level tracking with fork-level variance estimation. Within each JMH fork, it uses the naive estimator (MNP) to monitor the measurements collected so far and maintain the running fork mean. Across completed forks, it treats these fork means as approximately independent observations and estimates variance over them. Thus, m_{eff} is defined by the number of completed forks, and the corresponding budget n_{max} is interpreted on the fork scale. This design accounts for both within-fork measurements and across-fork replication, while remaining conservative under strong within-fork dependence.

5 Study Setup

To evaluate the performance of our approach, we conducted an empirical study in real-world Java projects. This section details the studied subjects, the execution environment, and the benchmark configuration to ensure reproducibility and practical relevance.

Table 3. Overview of the studied subjects. Contr. refers to the number of contributors. SLOC denotes the lines of source code.

Subject	Version	Commit SHA	Stars	Contr.	SLOC
RxJava	3.28.16	67e949db	48.1k	299	315,482
Eclipse Collections	12.0.0	b89e089b	2.5k	115	317,975
Zipkin	2.24.0	a36e0577	17.2k	156	39,915

5.1 Studied Subjects

In this work, we selected three real-world Java projects from different domains. The selected projects are RxJava [34], Eclipse Collections [11], and Zipkin [31]. These systems were chosen to reflect a broad spectrum of runtime characteristics, i.e., event-driven concurrency (RxJava), memory-intensive data structure operations (Eclipse-Collections), and I/O-heavy distributed tracing workloads (Zipkin). The heterogeneity of these subjects ensures that our empirical findings are robust and generalizable to a wide range of software engineering contexts. Detailed metadata, including repository size, commit history, and versioning, is summarized in Table 3.

5.2 Execution Environment and Benchmark Configuration

In the experimental setup, we forked JMH 1.37 and integrated the TT-MDE framework into its execution engine. All experiments were conducted on dedicated cloud VMs (Ubuntu 22.04 LTS, OpenJDK 21) provisioned with 4 vCPUs and 16GB RAM. In total, we evaluated 1,702 microbenchmarks. Unless stated otherwise, benchmarks ran in throughput mode with 1-second iterations and up to 5 forks. The baseline JMH configuration (STANDARD) used 50 warmup and 50 measurement iterations per fork. The stability-based approach (STABLE) replicated prior work [26] with up to 50 warmup iterations and 10 fixed measurement iterations per fork. TT-MDE uses no explicit warmup phase and permits up to 100 measurement iterations per fork, adaptively controlled by Algorithm 1. Unless noted otherwise, we use $\alpha = 0.05$ and $\beta = 0.2$, yielding a 5% false-positive bound for negligible regressions below δ_0 and 80% target power for regressions above δ_1 . To prevent premature stopping under transient noise perturbation, a flatness gate (sliding window size $w = 5$, threshold $\tau = 0.01$) must be satisfied before detectability-guided stopping decisions are enabled.

6 Results and Analyses

To systematically evaluate our approach in performance regression testing, this section addresses four research questions concerning result quality preservation, execution-cost reduction, practical regression-detection behavior, and statistical calibration.

6.1 RQ1: Does TT-MDE preserve benchmark result quality?

6.1.1 Motivation. TT-MDE introduces adaptive stopping decisions (*early stop* and *futile abort*) into the standard JMH execution model. Although designed to reduce runtime while maintaining statistical validity, adaptive stopping alters how samples are collected. This raises a fundamental concern: **Does TT-MDE change benchmark results compared to the standard JMH?**

6.1.2 Approach. To evaluate whether TT-MDE affects benchmark result quality, we compare standard JMH execution and TT-MDE

Table 4. A/A test and performance change rate results under different (δ_0, δ_1) configurations. For each metric, subscripts report the absolute difference relative to the STANDARD. Positive and negative subscripts indicate an increase and decrease compared to the STANDARD, respectively. All values are presented in percentage.

(δ_0, δ_1)	Method	A/A non-difference rate	Median change rate	#benches $\leq 1\%$	#benches $\leq 5\%$
(0.5%, 1%)	STANDARD	64.79	1.35	42.40	77.29
	MNP	59.19 _{-5.60}	1.46 _{+0.11}	41.09 _{-1.31}	74.86 _{-2.43}
	MAR1	60.11 _{-4.68}	1.48 _{+0.13}	38.57 _{-3.83}	74.69 _{-2.60}
	MNW	52.68 _{-12.12}	1.63 _{+0.28}	35.40 _{-7.00}	71.27 _{-6.02}
	MNF	56.95 _{-7.84}	1.65 _{+0.30}	34.21 _{-6.80}	70.21 _{-7.08}
(2.5%, 5%)	MNP	59.19 _{-5.60}	1.52 _{+0.16}	38.65 _{-3.75}	75.05 _{-2.24}
	MAR1	58.70 _{-6.09}	1.75 _{+0.40}	35.40 _{-7.00}	71.55 _{-5.74}
	MNW	53.52 _{-11.27}	1.75 _{+0.40}	35.40 _{-7.00}	71.55 _{-5.74}
	MNF	56.67 _{-8.12}	1.68 _{+0.33}	33.74 _{-8.66}	70.30 _{-6.99}
	MNP	58.72 _{-6.07}	1.52 _{+0.17}	38.18 _{-4.22}	74.58 _{-2.71}
(5%, 10%)	MAR1	59.17 _{-5.62}	1.57 _{+0.21}	36.69 _{-5.71}	74.69 _{-2.60}
	MNW	52.21 _{-12.59}	1.80 _{+0.45}	34.18 _{-8.22}	71.08 _{-6.21}
	MNF	57.33 _{-7.46}	1.66 _{+0.31}	33.46 _{-8.94}	70.58 _{-6.71}

execution using two complementary analyses: *statistical A/A test* and *performance change rate* analysis.

A/A Test. A/A test evaluates result consistency when no true performance difference exists [23]. We conduct two comparisons: (i) STANDARD vs. STANDARD, where two independent executions of standard JMH establish the intrinsic environmental noise floor; and (ii) STANDARD vs. TT-MDE, where a TT-MDE execution is compared against a standard execution to assess the impact of adaptive reconfiguration. Following performance engineering best practices [8, 25, 36], we estimate a 99% confidence interval for the ratio of mean throughput using bootstrap resampling (10,000 resamples) [15] with hierarchical sampling across forks and iterations for each benchmark pair. If the confidence interval contains 1, the executions are considered to be not significantly different. We report the proportion of such cases as the *A/A non-difference rate*.

Performance Change Rate. To quantify the effect magnitude, we compute the relative performance change rate for each benchmark: $\Delta = (\mu_{\text{TT-MDE}} - \mu_{\text{STANDARD}}) / \mu_{\text{STANDARD}}$, where $\mu_{\text{TT-MDE}}$ and μ_{STANDARD} is the mean value of collected JMH samples. We summarize the median change rate of Δ , and report the proportion of benchmarks whose absolute change remains within practical thresholds (i.e., 1% and 5%) [26].

6.1.3 Results and Analyses. TT-MDE largely preserves the benchmark outcomes close to the STANDARD behavior. Table 4 shows that in the STANDARD vs. STANDARD comparison, only 64.79% of benchmarks are considered as non-difference, reflecting substantial intrinsic variability. Against this noise floor, conservative variants (e.g., MNP and MAR1) remain close to STANDARD (≈ 59 –60%), MNF is slightly lower ($\approx 57\%$), and MNW shows the largest reduction among them (≈ 52 –54%). Overall, these deviations are only modest relative to STANDARD variability.

Most benchmarks exhibit small practical performance deviations. The median change rate of STANDARD is 1.35%. Across all TT-MDE variants and configurations, the median change rate remains low, ranging from 1.46% to 1.80%. The increases relative to STANDARD are modest for MNP and MAR1, and still limited for MNW and MNF. Overall, these results indicate that TT-MDE generally preserves practical result quality, with only small median deviations for most benchmarks.

Varying (δ_0, δ_1) has limited impact on result quality. Across all (δ_0, δ_1) settings, A/A non-difference rates remain stable for each method, showing that regression thresholds do not materially change outcomes. The consistent method ordering across settings confirms the stable and predictable behavior of TT-MDE.

RQ1 Takeaway: TT-MDE adaptive execution preserves benchmark result quality for most benchmarks, with conservative configurations exhibiting only limited variation. More aggressive configurations increase variability, while still keeping the majority of results within small practical change bounds.

6.2 RQ2: To what extent does TT-MDE reduce execution cost?

6.2.1 Motivation. Microbenchmark-based regression testing is often expensive in continuous integration environments. Standard JMH configurations allocate fixed numbers of forks and iterations to hedge against variance, which frequently leads to over-execution for low-variance benchmarks and insufficient statistical power for high-variance ones. Stability-based dynamic reconfiguration reduces redundant execution by stopping when measurements appear stable. However, stability criteria are independent of the target regression and provide no explicit guarantees regarding detectability, potentially stopping either too early or too late with respect to practically meaningful slowdowns. In contrast, TT-MDE monitors detectability online with respect to a predefined regression target and stops execution once the collected data either provides sufficient resolution to detect that target or indicates that achieving such resolution is unattainable within the remaining budget. This alignment raises a key practical question: **How much execution time can TT-MDE save relative to fixed and stability-based configurations while maintaining its statistical guarantees?**

6.2.2 Approach. To quantify execution-time reduction, we first measure intrinsic runtime overhead and then compare the total execution cost of TT-MDE and the stability-based approach (STABLE) with that of STANDARD JMH.

Runtime Overhead. We isolate the overhead of TT-MDE by executing the benchmark suite twice: once with STANDARD JMH and once with the TT-MDE implementation, while disabling *early stop* and *futile abort* in TT-MDE. This guarantees identical forks and iterations. Let t_b^{STANDARD} and $t_b^{\text{TT-MDE}}$ denote the end-to-end wall-clock execution time of benchmark b under STANDARD and TT-MDE implementations, respectively. The per-benchmark overhead is $o_b = t_b^{\text{TT-MDE}} / t_b^{\text{STANDARD}} - 1$. Execution time is measured via the JMH CLI and includes JVM startup and fork execution. Since fork and iteration counts are identical, o_b reflects only the cost of TT-MDE control logic. We summarize overall overhead using the median across benchmarks, as it is more robust than the mean to outliers and thus better reflects the typical overhead observed. Formally, $\bar{o} = \text{median}\{o_b \mid b \in \mathcal{B}\}$, where $\mathcal{B}$ denotes the set of benchmarks.

Execution Cost Comparison. We compare STANDARD, stability-based approach (STABLE), and TT-MDE under identical infrastructure and configuration (cf. Section 5.2), differing only in stopping policies. Let w_t and m_t denote the duration of one warmup and

Table 5. Relative execution time reductions (%) of STABLE and TT-MDE compared to STANDARD. Results are shown per subject and aggregated across subjects for different detectability configurations (δ_0, δ_1) . Higher values indicate greater reduction in execution cost.

Method	(δ_0, δ_1)	RxJava	Eclipse Collections	Zipkin	Total
STABLE	-	88.91	83.48	86.35	85.87
MNP	(0.5%, 1%)	89.87	93.16	91.45	91.71
	(2.5%, 5%)	91.16	85.52	91.91	88.18
	(5%, 10%)	94.22	92.21	95.48	93.21
MAR1	(0.5%, 1%)	84.01	89.75	90.59	87.42
	(2.5%, 5%)	87.65	75.09	89.89	81.07
	(5%, 10%)	89.33	83.26	94.14	86.34
MNW	(0.5%, 1%)	82.47	90.06	87.21	86.77
	(2.5%, 5%)	87.29	76.21	88.14	81.41
	(5%, 10%)	89.63	86.53	93.64	88.18
MNF	(0.5%, 1%)	91.81	89.90	90.60	90.73
	(2.5%, 5%)	89.58	89.12	89.88	89.35
	(5%, 10%)	91.12	87.88	88.39	89.25

Table 6. Decomposition of time reductions (%) into *early stop* and *futile abort* contributions. Higher values indicate greater reduction in execution cost.

Method	Termination State	(δ_0, δ_1)	RxJava	Eclipse Collections	Zipkin	Total
MNP	early stop	(0.5%, 1%)	69.36	73.01	71.69	70.55
		(2.5%, 5%)	88.28	84.60	88.67	86.35
		(5%, 10%)	89.18	85.02	88.67	86.94
	futile abort	(0.5%, 1%)	83.50	83.73	82.56	83.61
		(2.5%, 5%)	69.43	79.53	68.80	76.72
		(5%, 10%)	70.15	82.42	68.80	78.73
MAR1	early stop	(0.5%, 1%)	58.99	68.04	62.12	61.20
		(2.5%, 5%)	87.34	82.72	87.72	85.05
		(5%, 10%)	87.72	82.09	87.09	84.82
	futile abort	(0.5%, 1%)	79.64	77.82	80.57	78.41
		(2.5%, 5%)	50.74	53.97	27.40	52.39
		(5%, 10%)	60.80	60.80	0.00	57.86
MNW	early stop	(0.5%, 1%)	58.98	69.23	62.60	61.51
		(2.5%, 5%)	88.03	62.60	86.88	84.58
		(5%, 10%)	88.21	71.24	87.02	85.22
	futile abort	(0.5%, 1%)	80.78	80.91	83.32	81.02
		(2.5%, 5%)	58.31	68.00	74.20	65.09
		(5%, 10%)	59.87	71.24	74.20	69.83
MNF	early stop	(0.5%, 1%)	81.12	74.21	73.46	78.03
		(2.5%, 5%)	80.80	72.78	74.29	76.52
		(5%, 10%)	80.67	72.97	75.90	76.44
	futile abort	(0.5%, 1%)	74.42	72.43	74.42	73.40
		(2.5%, 5%)	72.31	70.49	76.58	71.21
		(5%, 10%)	67.68	71.12	81.40	70.31

one measurement iteration (both fixed to 1 second via JMH configuration). Let w_f and m_f denote the number of warmup and measurement iterations executed in fork f . For STANDARD with $f_{\max}$ forks, execution cost is $t_b^{\text{STANDARD}} = \sum_{f=1}^{f_{\max}} (w_f w_t + m_f m_t)$. For adaptive execution, the total cost depends on the forks and iterations actually executed: $t_b^D = \sum_{f \in \text{executed forks}} (w_f w_t + m_f m_t)$, where $D \in \{\text{STABLE}, \text{TT-MDE}\}$.

Time Reduction. For approach D , per-benchmark time reduction relative to STANDARD is $\text{TimeReduction}_b = 1 - t_b^D / t_b^{\text{STANDARD}}$. Per-subject results are computed as the mean of TimeReduction_b across benchmarks within each subject. To summarize the overall impact, we define the *Average Time Reduction (ATR)* across all subjects as $\text{ATR} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \text{TimeReduction}_b$. We further decompose savings by termination states (i.e., *early stop*, *futile abort*, *capped*) to quantify the contribution of each stopping mechanism.

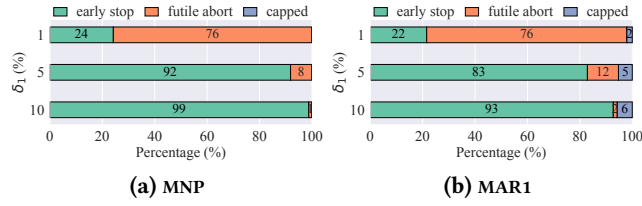


Figure 2. Distribution of termination states (*early stop*, *futile abort*, *capped*) across regression targets for MNP and MAR1. MNW and MNF show similar patterns; detailed results are available in the replication package.

6.2.3 Results and Analyses. TT-MDE introduces negligible end-to-end overhead. Across all MDE variants, the median end-to-end overhead remains small: 0.021% for MAR1, 0.020% for MNP, 0.011% for MNF, and 0.025% for MNW. The overhead distribution is concentrated near zero, with an interquartile range of [0.001%, 0.036%], and only 3.12% benchmarks exceeding 1% overhead. This indicates that the online variance estimation and stopping logic impose virtually no measurable performance penalty. Consequently, the substantial execution-time reductions reported below arise from adaptive termination rather than instrumentation overhead.

TT-MDE substantially reduces execution time compared to STANDARD. Table 5 reports relative execution-time reduction compared to the STANDARD. Across all subjects, TT-MDE consistently achieves large reductions in execution time. Aggregated across subjects, reductions range from 86.77% to 91.71% under the strictest regression target (0.5%, 1%), and from 86.34% to 93.21% under the loosest regression target (5%, 10%), depending on the MDE variant. Compared with STABLE, TT-MDE achieves comparable or higher reductions across most configurations. These results demonstrate that TT-MDE substantially reduces execution cost in practice.

Futile abort contributes substantially to time reduction under strict detectability targets. Table 6 and Figure 2 decompose time savings by termination states. For strict configurations $(\delta_0, \delta_1) = (0.5\%, 1\%)$, a large share of benchmarks terminate via *futile abort*, indicating that such small regressions are often difficult to detect within the available budget. These cases contribute substantially to time reduction by avoiding further sampling when the target is unlikely to be reached. As the regression target becomes looser, *early stop* becomes the dominant state, while *futile abort* becomes much less frequent. This pattern shows that *futile abort* is an important component of TT-MDE, especially under strict targets, where continuing execution would often consume additional time without improving detectability.

RQ2 Takeaway: TT-MDE achieves large and systematic execution-time reductions compared to STANDARD and STABLE, with negligible overhead.

6.3 RQ3: How does TT-MDE practically perform in regression detection?

6.3.1 Motivation. While TT-MDE is designed to provide detectability-guided decisions, its practical effectiveness must be evaluated under realistic regression scenarios. In real systems, performance regressions are heterogeneous and unpredictable: A single code change

may affect multiple benchmarks with varying magnitudes and noise characteristics. We therefore assess TT-MDE on mutation-induced regressions that mimic real performance bugs and compare its decisions against STANDARD execution as the reference. This raises the following question: **Does TT-MDE remain aligned with STANDARD decisions while reducing spurious alerts and execution cost under realistic regression injections?**

6.3.2 Approach. To evaluate TT-MDE in a realistic workflow, we create regression scenarios by injecting small, localized slowdowns into production code and comparing TT-MDE decisions against those of STANDARD and STABLE.

Regression injection via heavy-weight operator (HWO). We inject performance regressions using a lightweight mutation operator that inserts `Thread.sleep(0, 1)` at selected program locations (HWO). We choose `Thread.sleep(0, 1)` since it introduces a portable, non-optimizable delay that accumulates along hot paths, providing a stable way to emulate micro-regressions across environments. This strategy follows prior mutation-based performance regression evaluation practices [10, 17, 24].

Selecting mutation targets and affected benchmarks. To ensure that each injected regression is exercised during evaluation, we select mutation targets from methods that are invoked by the available microbenchmarks and restrict evaluation to benchmarks that call the mutated method either directly or transitively. For each subject, we execute all microbenchmarks with coverage instrumentation and compute, for every method in the source code, its *hit count*, i.e., the number of distinct benchmarks that execute it. To obtain mutation targets with diverse levels of benchmark exposure, we stratify methods by hit count. Concretely, we partition the hit-count range into up to 30 buckets using a step size of $\lceil \max(\text{hit})/30 \rceil$ and sample one mutation target from each non-empty bucket to obtain diversity across different levels of benchmark coverage. Specifically, we divide the observed hit-count range into up to 30 buckets using a step size of $\lceil \max(\text{hit})/30 \rceil$ and sample one method from each non-empty bucket. Since the hit-count distribution is uneven, some buckets remain empty and therefore contribute no target. This procedure yields 28 injected regressions for RxJava, 27 for Eclipse Collections, and 27 for Zipkin. In total, the evaluation involves 2,831 benchmark-mutant cases, including 1,102 from RxJava, 1,368 from Eclipse Collections, and 361 from Zipkin.

Oracle labeling via conservative bug-size estimation. Since the true slowdown magnitude varies across benchmarks even for the same injected delay, we derive a conservative per-benchmark “oracle” slowdown from the observed measurements. For each benchmark b , let b_A and b_B be the throughput samples from the baseline and mutant versions. We compute the throughput ratio $Q_b = \mu(b_B)/\mu(b_A)$ and estimate a 99% bootstrap confidence interval for Q_b using 10,000 resamples. We define the *bug size* as $\text{bug_size}(b) = 1 - U_{RCI}(Q_b)$, where $U_{RCI}(Q_b)$ is the upper endpoint of the 99% CI (conservative against over-claiming regressions). A benchmark is labeled as a *true regression* at threshold δ if $\text{bug_size}(b) \geq \delta$. This yields a threshold-specific oracle label without assuming that a mutation produces uniform slowdown across benchmarks.

Evaluation setup. For each benchmark pair (b_A, b_B) , we evaluate three execution policies: STANDARD, STABLE, and TT-MDE (MAR1).

Table 7. Comparison of stopping policies under different slowdown thresholds δ . Arrows indicate the preferred direction of each metric.

δ	Method	Precision $\uparrow$	Recall $\uparrow$	FAR $\downarrow$	AS $\uparrow$	UNR $\downarrow$
1%	STANDARD	74.20	87.23	19.30	72.78	41.77
	STABLE	81.38	66.47	9.68	47.48	70.14
	TT-MDE ($\delta_0 = 0.1\%$)	88.36	79.08	6.63	53.82	67.14
	TT-MDE ($\delta_0 = 0.01\%$)	86.46	80.50	8.02	56.80	63.90
	TT-MDE ($\delta_0 = 0.001\%$)	85.78	80.59	8.50	56.99	63.86
5%	STANDARD	58.35	98.75	32.18	98.73	1.69
	STABLE	65.38	95.10	22.99	88.95	17.63
	TT-MDE ($\delta_0 = 0.1\%$)	78.89	97.81	11.95	95.98	6.02
	TT-MDE ($\delta_0 = 0.01\%$)	77.86	97.91	12.71	96.08	5.97
	TT-MDE ($\delta_0 = 0.001\%$)	77.80	97.91	12.75	95.95	6.19
10%	STANDARD	54.31	100.00	34.45	99.58	0.84
	STABLE	61.07	98.65	25.75	98.46	2.11
	TT-MDE ($\delta_0 = 0.1\%$)	76.29	99.55	12.67	99.93	0.11
	TT-MDE ($\delta_0 = 0.01\%$)	75.06	99.55	13.54	99.96	0.05
	TT-MDE ($\delta_0 = 0.001\%$)	74.87	99.55	13.68	99.96	0.05

We use MAR1 as the representative TT-MDE variant because earlier results show that it provides a good balance between result quality and execution savings. All approaches apply the same statistical regression test (cf. Section 4.2) to the collected measurements and evaluate the same regression thresholds $\delta \in \{1\%, 5\%, 10\%\}$. For TT-MDE, we use $(0.1\%, \delta)$ as the primary near-zero approximation and additionally evaluate $\delta_0 \in \{0.01\%, 0.001\%\}$ for sensitivity analysis. We keep δ_0 independent of δ_1 , since scaling it with δ_1 would change the negligible-effect region and the resulting decision semantics. All configurations use the same benchmark pairs, oracle labels, and regression-analysis procedure. This setup aims to make the methods as comparable as possible while keeping their decision semantics closely aligned.

Metrics. We evaluated stopping policies along two complementary dimensions: (i) regression detection quality, (ii) statistical adequacy of the stopping point.

Detection quality. We report **Precision**, **Recall**, and **False Alert Rate (FAR)** to evaluate the reliability of regression alerts. Precision measures how often detected regressions correspond to oracle true regressions, while FAR quantifies the rate of false alarms among non-regression cases.

Stopping adequacy. Since our goal is to stop only when sufficient statistical resolution has been achieved, we also evaluate whether the stopping point provides adequate detectability. Let $\widehat{MDE}_{stop}$ denote the minimum detectable effect estimated at stopping time. We define **Adequate Stop (AS)** as the fraction of cases where $\widehat{MDE}_{stop} \leq \delta_1 - \delta_0$, indicating that the collected measurements are sufficient to detect the target slowdown. We further report **Underpowered Non-Regression (UNR)**, which captures cases where a method concludes that no regression exists even though $\widehat{MDE}_{stop} \geq \delta_1 - \delta_0$. Such cases represent *false-confidence* non-regression decisions made without sufficient statistical power.

6.3.3 Results and Analyses. TT-MDE improves regression decision quality over STABLE while remaining broadly aligned with STANDARD. Table 7 shows that TT-MDE consistently achieves higher precision and recall than STABLE across all slowdown thresholds. Relative to STANDARD, TT-MDE attains higher precision and lower false-alert rates across all thresholds, while recall remains comparable at 5% and 10%. Overall, these results indicate that detectability-guided stopping improves regression reliability over

STABLE while preserving decisions broadly consistent with the STANDARD reference.

TT-MDE yields more statistically adequate stopping decisions than STABLE. TT-MDE consistently achieves higher *Adequate Stop* rates and markedly lower *Underpowered Non-Regression* rates than STABLE across all thresholds, indicating that its decisions are made with stronger statistical support at stopping time. Compared with STANDARD, TT-MDE shows lower *Adequate Stop* at 1% and 5%, while reaching a comparable level at 10%. These results show that detectability-guided stopping substantially reduces *Underpowered Non-Regression* decisions, especially at moderate and larger slowdown thresholds.

The near-zero approximation has limited impact on the results. Reducing δ_0 from 0.1% to 0.001% modestly increases its false alert rate. The largest FAR increase is 1.87%, from 6.63% to 8.50% at $\delta_1 = 1\%$. TT-MDE retains a lower FAR than STABLE for every evaluated target and δ_0 setting. Precision, recall, and stopping-adequacy results also remain qualitatively consistent, showing that the primary conclusions are not driven by the choice of $\delta_0 = 0.1\%$.

RQ3 Takeaway: TT-MDE reduces false alerts and preserves most regression and non-regression decisions while producing more statistically adequate stopping decisions than STABLE.

6.4 RQ4: Does TT-MDE provide calibrated detectability guarantees under realistic measurement noise?

6.4.1 Motivation. RQ4 evaluates whether TT-MDE empirically satisfies the statistical guarantees defined in Section 4.2. Specifically, the two-threshold design provides two theoretical properties: (i) bounded false-positive rate for negligible regressions ($\Delta \leq \delta_0$), and (ii) target detection power for meaningful regressions ($\Delta \geq \delta_1$). RQ4 therefore performs a calibration experiment to verify whether the observed regression-detection behavior under realistic benchmark noise matches these theoretical guarantees.

6.4.2 Approach. To assess whether TT-MDE satisfies its intended false-positive and power guarantees under realistic noise, we adopt a semi-simulation framework that preserves real measurement variability while enabling precise control over regression magnitude.

Why Semi-Simulation? Evaluating statistical guarantees requires knowledge of the true regression magnitude. In real codebases, regression size is neither controllable nor directly observable: A single code change may induce heterogeneous and unpredictable effects across benchmarks, even when they exercise similar execution paths. As a result, code-level slowdown injection cannot produce uniform or precisely controlled regression magnitudes across all benchmarks. Consequently, purely empirical executions cannot determine whether a regression should be statistically detectable under a given configuration. Moreover, a fully empirical calibration study is also computationally infeasible at our scale. In RQ3, we already evaluated 2,831 microbenchmarks. Under the realistic benchmark configuration used in this paper, each benchmark requires approximately 500 seconds of execution. Calibrating statistical guarantees would further require evaluating multiple target regression magnitudes, repeating each configuration across Monte Carlo trials,

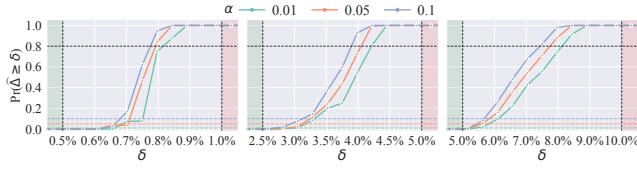


Figure 3. Empirical false positive behavior. The y-axis shows the probability of reporting a regression, i.e., the empirical false-positive rate. Curves vary the target α , and horizontal dashed lines indicate the nominal false-positive bound α . The light-green region denotes the false-positive-controlled zone ($\delta \leq \delta_0$), where TT-MDE is designed to keep the reporting probability at or below α . Results show that the empirical behavior closely follows this intended bound, while no false-positive guarantee is provided above δ_0 by design.

and executing paired regression runs. With 3 regression targets, 30 trials [2, 25], $\alpha \in \{0.01, 0.05, 0.1\}$, $\beta \in \{0.05, 0.1, 0.2\}$, the total execution time would be approximately 4.05 years. Such cost makes exhaustive empirical calibration impractical. Semi-simulation is therefore not merely a convenience, but a necessary design choice for validating calibrated detectability guarantees under realistic benchmark noise.

Trace Collection and Injection. We collect measurement traces by running the TT-MDE measurement acquisition procedure, ensuring that calibration is evaluated under the same sampling pipeline used in practice. These traces retain fork- and iteration-level variability, JVM effects, and cloud-induced noise.

To simulate a relative slowdown $\Delta \geq 0$, we inject controlled regressions directly into throughput observations. Let $x_{v,i}$ denote the i -th throughput measurement. The injected trace is constructed as $x'_{v,i} = x_{v,i}(1 - \Delta)$, which multiplicatively scales throughput while preserving the original noise realization. The same underlying trace is reused across different Δ_{true} values to ensure comparable stochastic conditions.

Empirical Calibration. For each configuration $(\delta_0, \delta_1, \alpha, \beta)$, we apply the TT-MDE to the semi-simulated traces. To cover a broad range of true effect sizes, we draw injected slowdowns Δ uniformly from the interval $[0, 0.15]$. To evaluate false-positive calibration, we fixed $\beta = 0.2$ and vary $\alpha \in \{0.01, 0.05, 0.10\}$. To evaluate empirical power, we fixed $\alpha = 0.05$ and vary $\beta \in \{0.05, 0.10, 0.20\}$. For each injected slowdown Δ , we estimate the empirical detection probability $P(\hat{\Delta}) = \Pr(\text{REPORTREGRESSION} \mid \hat{\Delta})$. We compare the resulting detection curves against the theoretical guarantees defined in Section 4.2, verifying (i) false-positive control below δ_0 and (ii) power around δ_1 .

6.4.3 Results and Analyses. TT-MDE maintains empirical false-positive control below δ_0 across all configurations. Figure 3 reports the probability of flagging a regression as the true slowdown Δ varies. For $\Delta \leq \delta_0$, empirical flagging probabilities closely match the nominal α levels across all threshold settings. The curves remain flat in the false-positive-controlled region and align with the dashed reference lines. This confirms that the lower threshold δ_0 successfully enforces a bounded false-positive rate even under sequential stopping and realistic JMH noise.

Empirical power above δ_1 matches the target $1 - \beta$. Figure 4 presents empirical detection probabilities under varying β . Once $\Delta \geq \delta_1$, detection probability increases sharply and stabilizes

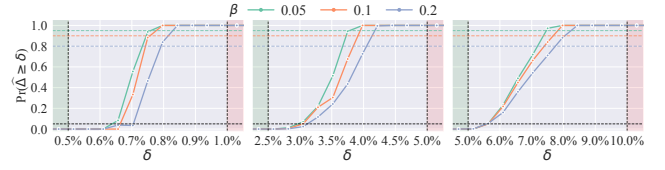


Figure 4. Empirical power behavior. The y-axis shows the probability of reporting a regression, i.e., empirical power. Curves vary the target β , and horizontal dashed lines indicate the nominal target power $1 - \beta$. The light-red region marks the power-guaranteed zone ($\delta \geq \delta_1$), where TT-MDE is designed to achieve the target power. The results show that the empirical detection probability closely tracks this guarantee, while no power guarantee is provided below δ_1 by design.

near the nominal power levels. Smaller β (higher target power) produces correspondingly higher empirical curves, and larger β produces lower ones, demonstrating tight alignment with specified guarantees. The transition to full power occurs rapidly after crossing δ_1 , indicating that the upper threshold correctly delineates the power-guaranteed region.

RQ4 Takeaway: TT-MDE achieves calibrated detectability under realistic measurement noise: false positives remain bounded by α , power above δ_1 matches $1 - \beta$. This demonstrates that TT-MDE delivers principled statistical guarantees.

7 Discussion

This section discusses the configuration and robustness of TT-MDE.

7.1 Impact of Non-Gaussian Measurement Noise

TT-MDE uses Gaussian quantiles to approximate the sampling distribution of the mean-based regression estimator. This approximation may become optimistic when the effective sample size is small or the measurements are severely heavy-tailed. Although the flatness gate and the Phase 2 detectability check help prevent premature decisions, they cannot guarantee calibrated error rates or target power under extreme non-Gaussian noise.

To examine this limitation, we conduct a robustness analysis using the collected throughput traces. For each observation x_i , we define $\tilde{x}_i = \frac{x_i}{1 + \epsilon_i}$, where $\epsilon_i = 0$ with probability $1 - p$ and $\ln \epsilon_i \sim \mathcal{N}(-2, 1)$ otherwise. This models one-sided events such as garbage-collection pauses, scheduler stalls, and cloud noise. We evaluate MAR1 with $(\delta_0, \delta_1) = (2.5\%, 5\%)$, $\alpha = 0.05$, and $1 - \beta = 0.8$, while varying the perturbation probability p .

Rare non-Gaussian perturbations have little effect, whereas frequent perturbations substantially reduce detection power. As shown in Table 8, $p = 0$ represents the original traces, under

Table 8. Statistical power under different frequencies of log-normal perturbations. The empirical false-positive rate is omitted since it is 0 for every subject and setting.

p	RxJava	Eclipse Collections	Zipkin	Overall
0.00	0.921	0.853	0.966	0.887
0.05	0.916	0.839	0.966	0.878
0.50	0.606	0.417	0.650	0.507
1.00	0.273	0.146	0.137	0.198

Table 9. Sensitivity of the flatness-gated threshold τ .

τ	A/A non-difference rate	Median change rate	TimeSAVING
0.001	58.70%	1.51%	73.48%
0.050	58.42%	2.06%	86.83%
0.100	58.80%	2.13%	87.86%

which TT-MDE meets its configured false-positive and power targets. Increasing the perturbation probability to $p = 0.05$ reduces overall power only slightly, from 0.887 to 0.878. In contrast, power falls to 0.507 at $p = 0.5$ and 0.198 at $p = 1$, well below the target of 0.8. The empirical false-positive rate remains 0 for every subject and setting. Thus, under the evaluated configuration, severe heavy-tailed noise mainly compromises detectability rather than increasing false alerts. In environments with persistent non-Gaussian noise, TT-MDE may therefore fail to achieve its target power and should be combined with a more robust uncertainty estimator.

7.2 Sensitivity Analysis of the Flatness-Gated Threshold

The flatness gate is jointly controlled by the window size w and the threshold τ . Since activation requires $\max(H) - \min(H) \leq \tau$ over the latest w checkpoints, a larger window generally yields a wider observed range and may require a larger τ to maintain comparable activation strictness. To isolate the effect of the permitted fluctuation in the recent MDE trajectory, we therefore fix $w = 5$ and vary only τ . For simplicity, we evaluate only the MAR1 variant.

A stricter flatness threshold reduces execution-time savings while lowering the observed median change rate. As shown in Table 9, decreasing τ from 0.100 to 0.001 reduces time savings from 88.24% to 74.59%, while the median change rate decreases from 1.94% to 1.39%. The A/A non-difference rate also increases slightly, from 61.50% to 62.34%. These results indicate that a smaller τ delays adaptive termination and increases measurement effort.

The aggregate A/A outcome is limited affected by τ . Across the evaluated thresholds, the A/A non-difference rate varies by less than one percentage point. Thus, τ mainly controls the aggressiveness of adaptive termination and the resulting execution cost, while having limited influence on the aggregate A/A result.

7.3 Practical Configuration of Detectability Thresholds

The two-threshold specification (δ_0, δ_1) defines both the detectability contract and the measurement effort required by TT-MDE. In practice, δ_0 should represent the largest slowdown considered negligible, based on historical variability or engineering tolerance, whereas δ_1 should represent the smallest actionable regression. For example, 5% or 10% may be suitable for routine CI, while latency- or cost-sensitive systems may require smaller thresholds.

The gap $\delta_1 - \delta_0$ determines the required statistical resolution. A narrower gap requires more effective samples and higher execution cost, whereas a wider gap enables earlier stopping and larger time savings. Threshold selection is therefore an explicit engineering trade-off between sensitivity and cost.

A *futile abort* indicates that the requested resolution is unlikely to be achieved under the observed variance and execution budget; it should not be interpreted as evidence of no regression. Frequent

futile aborts instead signal that practitioners may need to increase the budget, improve environmental isolation, redesign noisy benchmarks, or widen the indifference region. Thus, futility serves not only as a cost-saving mechanism, but also as a diagnostic of benchmark adequacy and target feasibility.

8 Threats to Validity

We discuss the threats to the validity of our approach.

External Validity. Our evaluation focuses on three widely used Java systems that rely on JMH-based microbenchmarks. Although these projects reflect realistic performance engineering practice, results may differ for other programming languages, benchmarking frameworks, or hardware architectures. We inject regressions using delay-based mutation operators grounded in prior work [10, 17, 24]. While this approach enables controlled experimentation, it does not cover all regression types (e.g., cache-related or algorithmic changes). Extending regression models to additional mutation categories will be done in future work.

Internal Validity. Microbenchmark measurements are sensitive to environmental noise, runtime dynamics, and non-stationarity. We mitigate these risks using standard benchmarking practices. Our adaptive stopping rule depends only on variance estimates and not on cross-version mean differences, reducing the risk of sequential stopping bias [20]. However, extreme non-Gaussian behavior or heavy-tailed distributions may affect finite-sample accuracy. Our semi-simulation preserves the observed benchmark noise in the collected traces, but assumes that this noise remains unchanged after regression injection. In practice, real regressions may also alter the noise distribution over time, for example by changing contention or GC dynamics. Such distribution shift is not captured by our simulation and may affect the empirical calibration results.

Construct Validity. We evaluate regression detectability using relative throughput degradation and predefined thresholds (e.g., 1%, 5%). While common in engineering practice, these thresholds may not suit all domains, and throughput alone may miss effects such as garbage collection or I/O variability. Richer runtime profiling could improve detectability modeling. Our conservative oracle defines regressions using the upper bound of a 99% bootstrap confidence interval, reducing false labels but potentially classifying some high-variance injected slowdowns as non-regressions. Thus, reported recall may underestimate recall relative to the injected slowdown.

9 Conclusions

We introduce a detectability-guided framework for microbenchmark-based regression testing that replaces stability-oriented stopping with statistically calibrated execution control. By integrating on-line variance estimation with a two-threshold MDE formulation, our approach dynamically adapts sampling efforts to the practical significance of performance changes. Empirical results show that detectability-guided stopping substantially reduces execution cost while maintaining reliable regression decisions. By treating detection capability, not measurement stability, as the primary objective, this work establishes a principled foundation for adaptive and scalable microbenchmark regression testing.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. This work received funding from the Federal Ministry of Research, Technology and Space (BMFTR) and the State of Berlin under grant number 16DII141 (Weizenbaum Institute) and by the German Research Foundation (DFG) under project number 460234259 (NFDI4DataScience).

Data-Availability Statement

The replication package of this study is available at: <https://doi.org/10.5281/zenodo.21760554> [5]

References

- [1] Hammam M Alghamdi, Mark D Syer, Weiyei Shang, and Ahmed E Hassan. 2016. An automated approach for recommending when to stop performance tests. In *2016 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE, 279–289. doi:10.1109/ICSME.2016.46
- [2] Lubomír Bulej, Tomáš Bureš, Vojtěch Horký, Jaroslav Kotráž, Lukáš Marek, Tomáš Trojáněk, and Petr TžMa. 2017. Unit testing performance with stochastic performance logic. *Automated Software Engineering* 24, 1 (2017), 139–187. doi:10.1007/s10515-015-0188-0
- [3] Yen Chang, Tianhao Song, Jane Monaco, and Anastasia Ivanova. 2020. Futility stopping in clinical trials, optimality and practical considerations. *Journal of biopharmaceutical statistics* 30, 6 (2020), 1050–1059. doi:10.1080/10543406.2020.1818253
- [4] Jinfu Chen, Weiyei Shang, and Emad Shihab. 2022. PerfJIT: Test-Level Just-in-Time Prediction for Performance Regression Introducing Commits. *IEEE Trans. Softw. Eng.* 48, 5 (May 2022), 1529–1544. doi:10.1109/TSE.2020.3023955
- [5] Zongxiang Chen, Derui Zhu, Jinfu Chen, Kundi Yao, Alexander Pretschner, Weiyei Shang, Manfred Hauswirth, and Sonja Schimmler. 2026. *Reproduction Package for Article "Stop When It Matters: Detectability-Guided Microbenchmarking for Performance Regression Testing"*. doi:10.5281/zenodo.21760554
- [6] Mike Chow, Yang Wang, William Wang, Ayichew Hailu, Rohan Bopadikar, Bin Zhang, Jialiang Qu, David Meisner, Santosh Sonawane, Yunqi Zhang, Rodrigo Paim, Mack Ward, Ivor Huang, Matt McNally, Daniel Hodges, Zoltan Farkas, Caner Gocmen, Elvis Huang, and Chunqiang Tang. 2024. ServiceLab: Preventing Tiny Performance Regressions at Hyperscale through Pre-Production Testing. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 545–562. <https://www.usenix.org/conference/osdi24/presentation/chow>
- [7] Shein-Chung Chow and Mark Chang. 2008. Adaptive design methods in clinical trials—a review. *Orphanet journal of rare diseases* 3, 1 (2008), 11. doi:10.1186/1750-1172-3-11
- [8] Christoph Laaber. 2020. pa - Performance (Change) Analysis using Bootstrap. <https://github.com/chrstphlbr/pa>. Accessed: 2026-02-14.
- [9] Charlie Curtsinger and Emery D. Berger. 2013. STABILIZER: statistically sound performance evaluation. *SIGARCH Comput. Archit. News* 41, 1 (March 2013), 219–228. doi:10.1145/2490301.2451141
- [10] Pedro Delgado-Pérez, Ana Belén Sánchez, Sergio Segura, and Inmaculada Medina-Bulo. 2021. Performance mutation testing. *Software Testing, Verification and Reliability* 31, 5, Article e1728 (2021). doi:10.1002/stvr.1728
- [11] Eclipse Collections Project. 2024. Eclipse Collections. Eclipse Foundation, <https://github.com/eclipse-collections/eclipse-collections/commit/b89e089b895c6197fc54cf932c108071a0df7316>. Version 12.0.0-SNAPSHOT; commit b89e089b895c6197fc54cf932c108071a0df7316; accessed 2025-07-18.
- [12] Ping Gao, James H Ware, and Cyrus Mehta. 2008. Sample size re-estimation for adaptive sequential design in clinical trials. *Journal of biopharmaceutical statistics* 18, 6 (2008), 1184–1196. doi:10.1080/10543400802369053
- [13] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically rigorous java performance evaluation. *ACM SIGPLAN Notices* 42, 10 (2007), 57–76. doi:10.1145/1297105.1297033
- [14] Martin Grambow, Denis Kovalev, Christoph Laaber, Philipp Leitner, and David Bermbach. 2023. Using microbenchmark suites to detect application performance changes. *IEEE Transactions on Cloud Computing* 11, 3 (2023), 2575–2590. doi:10.1109/TCC.2022.3217947
- [15] Tim C. Hesterberg. 2015. What teachers should know about the bootstrap: Resampling in the undergraduate statistics curriculum. *The American Statistician* 69, 4 (2015), 371–386. doi:10.1080/00031305.2015.1089789
- [16] Peng Huang, Xiao Ma, Dongcai Shen, and Yuanyuan Zhou. 2014. Performance regression testing target prioritization via performance risk analysis. In *Proceedings of the 36th International Conference on Software Engineering*. 60–71. doi:10.1145/2568225.2568232
- [17] Mostafa Jangali, Yiming Tang, Niclas Alexandersson, Philipp Leitner, Jinjui Yang, and Weiyei Shang. 2023. Automated generation and evaluation of JMH microbenchmark suites from unit tests. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1704–1725. doi:10.1109/TSE.2022.3188005
- [18] Nils Japke, Martin Grambow, Christoph Laaber, and David Bermbach. 2025. μ OpTime: Statically Reducing the Execution Time of Microbenchmark Suites Using Stability Metrics. *ACM Trans. Softw. Eng. Methodol.* 34, 8, Article 231 (Oct. 2025), 26 pages. doi:10.1145/3715322
- [19] Guoliang Jin, Linhai Song, Xiaoming Shi, Joel Scherpelz, and Shan Lu. 2012. Understanding and detecting real-world performance bugs. *ACM SIGPLAN Notices* 47, 6 (2012), 77–88. doi:10.1145/2345156.2254075
- [20] Ramesh Johari, Pete Koomen, Leonid Pekelis, and David Walsh. 2022. Always valid inference: Continuous monitoring of a/b tests. *Operations Research* 70, 3 (2022), 1806–1821. doi:10.1287/opre.2021.2135
- [21] Tomas Kalibera and Richard Jones. 2013. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 international symposium on memory management*. 63–74. doi:10.1145/2464157.2464160
- [22] Meinhard Kieser and Tim Friede. 2003. Simple procedures for blinded sample size adjustment that do not affect the type I error rate. *Statistics in medicine* 22, 23 (2003), 3571–3581. doi:10.1002/sim.1585
- [23] Ron Kohavi and Roger Longbotham. 2011. Unexpected results in online controlled experiments. *Acm Sigkdd Explorations Newsletter* 12, 2 (2011), 31–35. doi:10.1145/1964897.1964905
- [24] Christoph Laaber and Philipp Leitner. 2018. An evaluation of open-source software microbenchmark suites for continuous performance assessment. In *Proceedings of the 15th International Conference on Mining Software Repositories*. 119–130. doi:10.1145/3196398.3196407
- [25] Christoph Laaber, Joel Scheuner, and Philipp Leitner. 2019. Software microbenchmarking in the cloud. How bad is it really? *Empirical Software Engineering* 24, 4 (2019), 2469–2508. doi:10.1007/s10664-019-09681-1
- [26] Christoph Laaber, Stefan Würsten, Harald C. Gall, and Philipp Leitner. 2020. Dynamically reconfiguring software microbenchmarks: Reducing execution time without sacrificing result quality. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 989–1001. doi:10.1145/3368089.3409683
- [27] Shaikh Mostafa, Xiaoyin Wang, and Tao Xie. 2017. PerfRanker: prioritization of performance regression tests for collection-intensive software. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (Santa Barbara, CA, USA) (ISSTA 2017)*. Association for Computing Machinery, New York, NY, USA, 23–34. doi:10.1145/3092703.3092725
- [28] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F Sweeney. 2009. Producing wrong data without doing anything obviously wrong! *ACM Sigplan Notices* 44, 3 (2009), 265–276. doi:10.1145/1508284.1508275
- [29] Whitney K Newey and Kenneth D West. 1986. A simple, positive semi-definite, heteroskedasticity and autocorrelationconsistent covariance matrix. (1986). doi:10.3386/t0055
- [30] OpenJDK. 2024. Java Microbenchmark Harness (JMH). <https://github.com/openjdk/jmh>. Accessed: 2025-07-18.
- [31] OpenZipkin. 2024. Zipkin. <https://github.com/openzipkin/zipkin>. Accessed: 2025-07-18.
- [32] Stuart J Pocock. 1977. Group sequential methods in the design and analysis of clinical trials. *Biometrika* 64, 2 (1977), 191–199. doi:10.1093/biomet/64.2.191
- [33] Martin Posch, Florian Klingmueller, Franz König, and Frank Miller. 2018. Estimation after blinded sample size reassessment. *Statistical Methods in Medical Research* 27, 6 (2018), 1830–1846. doi:10.1177/0962280216670424
- [34] ReactiveX. 2024. RxJava. <https://github.com/ReactiveX/RxJava>. Accessed: 2025-07-18.
- [35] David Georg Reichelt and Stefan Kühne. 2018. Better early than never: Performance test acceleration by regression test selection. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 127–130. doi:10.1145/3185768.3186289
- [36] Marcelino Rodriguez-Cancio, Benoit Combemale, and Benoit Baudry. 2016. Automatic microbenchmark generation to prevent dead code elimination and constant folding. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 132–143. doi:10.1145/2970276.2970346
- [37] Donald J Schuurmann. 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of pharmacokinetics and biopharmaceutics* 15, 6 (1987), 657–680. doi:10.1007/BF01068419
- [38] Weichung Joe Shih. 2006. Group sequential, sample size re-estimation and two-stage adaptive designs in clinical trials: a comparison. *Statistics in Medicine* 25, 6 (2006), 933–941. doi:10.1002/sim.2252
- [39] Richard Simon. 1989. Optimal two-stage designs for phase II clinical trials. *Controlled clinical trials* 10, 1 (1989), 1–10. doi:10.1016/0197-2456(89)90015-9
- [40] Steven Snapinn, Mon-Gy Chen, Qi Jiang, and Tony Koutsoukos. 2006. Assessment of futility in clinical trials. *Pharmaceutical Statistics: The Journal of Applied*

- Statistics in the Pharmaceutical Industry* 5, 4 (2006), 273–281. doi:10.1002/pst.216
- [41] Antonio Solanas, Rumen Manolov, and Vicenta Sierra. 2010. Lag-one autocorrelation in short series: Estimation and hypotheses testing. *Psicológica* 31, 2 (2010), 357–381. <https://www.redalyc.org/pdf/169/16917017009.pdf>
 - [42] Gernot Wassmer and Werner Brannath. 2016. *Group sequential and confirmatory adaptive designs in clinical trials*. Vol. 301. Springer. doi:10.1007/978-3-319-32562-0
 - [43] Janet Wittes and Erica Brittain. 1990. The role of internal pilot studies in increasing the efficiency of clinical trials. *Statistics in medicine* 9, 1-2 (1990), 65–72. doi:10.1002/sim.4780090113
 - [44] Jialin Xu, Charles Audet, Charles E DiLiberti, Walter W Hauck, Timothy H Montague, Alan F Parr, Diane Potvin, and Donald J Schuirmann. 2016. Optimal adaptive sequential designs for crossover bioequivalence studies. *Pharmaceutical Statistics* 15, 1 (2016), 15–27. doi:10.1002/pst.1721
 - [45] Dong Young Yoon, Yang Wang, Miao Yu, Elvis Huang, Juan Ignacio Jones, Abhinay Kukkadapu, Osman Kocas, Jonathan Wiepert, Kapil Goenka, Sherry Chen, Yanjun Lin, Zhihui Huang, Jocelyn Kong, Michael Chow, and Chunqiang Tang. 2024. FBDetect: Catching Tiny Performance Regressions at Hyperscale through In-Production Monitoring. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 522–540. doi:10.1145/3694715.3695977

Received 2026-03-27; accepted 2026-06-18